

How University Students Read Non-English Code

Rachel Wilson
Carnegie Mellon University

Motivation

Computer programming is **English-centric**, meaning non-native speakers face barriers when learning to program [1][4].

```
18 定义 转换输出(数目):  
19     结果 = 1  
20     循环 (数目 // 100) 里的 i:  
21         结果 = 结果 * 3  
22     返回 结果  
23
```

```
def transformOutput(num):  
    result = 1  
    for i in range(num//5):  
        result = result * 2  
    return result
```

Should programming languages be translated or should programmers be taught English?

Research Focus

In this case study, I research how English-speaking university students leverage their diverse linguistic and programming backgrounds to read non-English code.

Goals

- 1. Identify what skills students use to read non-English code
- 2. Reflect on localized vs. English programming languages
- 3. Formulate further research questions

Approach

First, understand linguistic and programming backgrounds through a **survey**.
Next, observe students in **interviews with a code tracing activity**.

References

[1] Philip J. Guo. Non-native english speakers learning computer programming: Barriers, desires, and design opportunities. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, page 1–14, 2018.

[2] Chris Piech and Sami Abu-El-Haija. Human languages in source code: Auto-translation for localized instruction. In *Proceedings of the Seventh ACM Conference on Learning @ Scale*, page 167–174, 2020.

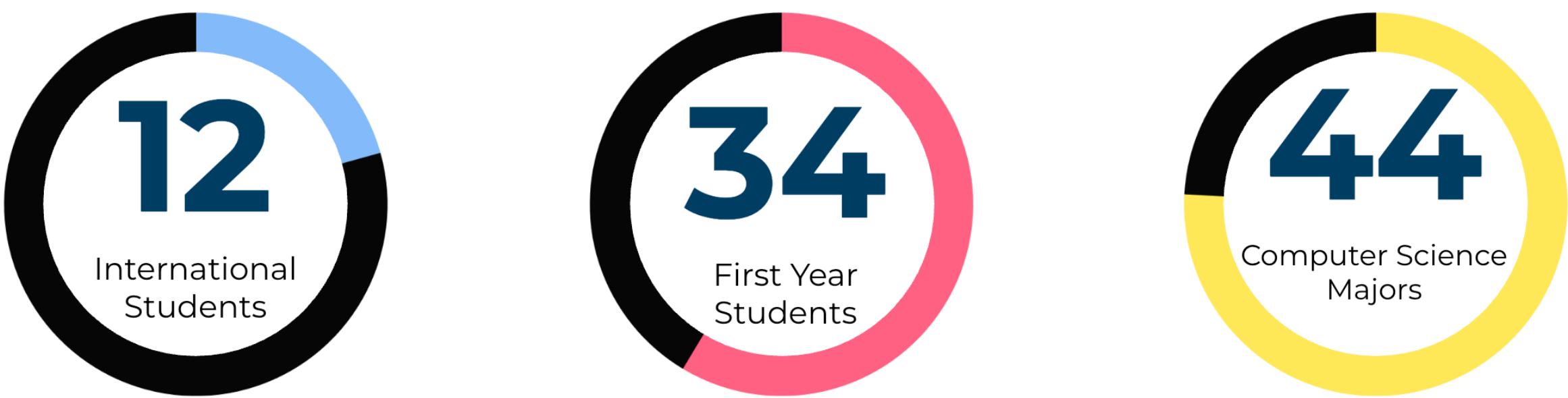
[3] Adalbert Gerald Soosai Raj, Eda Zhang, Saswati Mukherjee, Jim Williams, Richard Halverson, and Jignesh M. Patel. Effect of native language on student learning and classroom interaction in an operating systems course. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education*, page 499–505, 2019.

[4] Alaaeddin Swidan and Felienne Hermans. A framework for the localization of programming languages. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on SPLASH-E*, page 13–25, 2023.

THE SURVEY: Linguistic and Coding Backgrounds

Demographics

58 undergraduate students responded to the survey.



Language Backgrounds

Students listed **21 distinct languages** as "Known Well" (≤ 3 per student)
Top 5: English (58) Spanish (14) Mandarin (13) Hindi (5) French (4)
A large majority (42) were at least bilingual, showing bias towards multilingual students.

Coding Backgrounds

45 students indicated prior coding experience

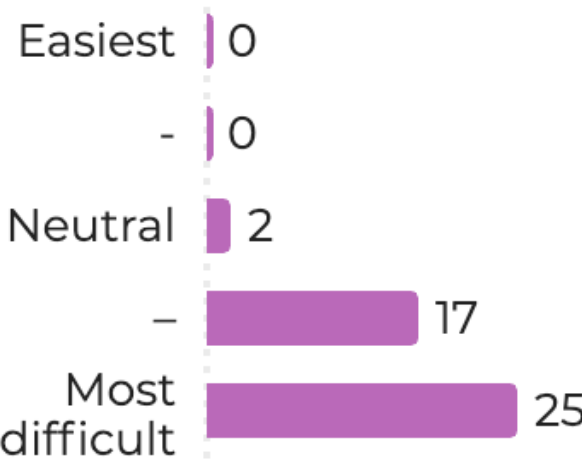


When learning to code, the **language of instruction** was more limited than the languages spoken by the students at the time.

Considerations for the Interviews

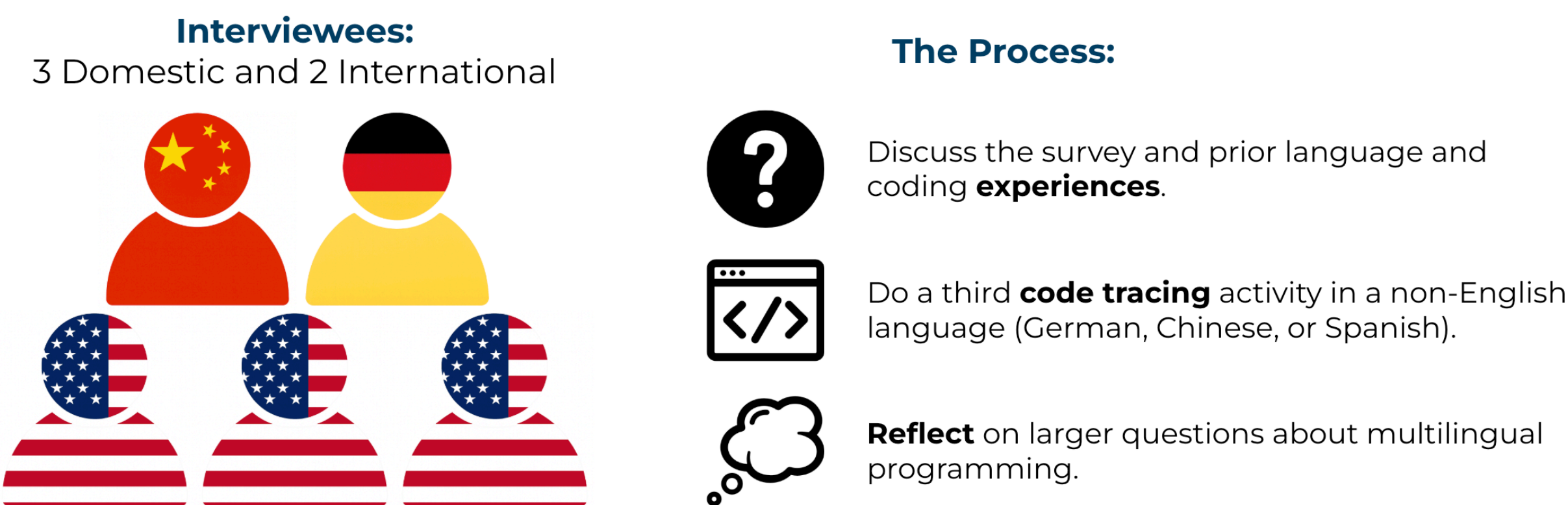
Students agreed it would be more difficult to program in a language they do not speak.

There was no consensus on if translating or teaching English would make programming more accessible to non-English speakers.



THE INTERVIEW: Coding Activity and Reflections

Methodology



Student Insights

"The most difficult part of [learning to code] is juggling all of the surface level knowledge, like what tools do I have access to?"

"I have to know English to be able to collaborate with others."

"I feel like it can be beneficial if we teach the core concepts of computer science at an early stage in the native language that the speakers speak."

Interview Results

- 1. Syntax was one of the hardest parts of learning to program.
- 2. Patterns in code syntax allowed students to confidently guess semantics.
- 3. The human language of the code evoked emotional reactions for the programmer.

Translate or Educate?

Localization of programming languages and support for non-English language features could increase engagement for multilingual students and lower the barrier of entry to computer programming around the world. [2][3][4]

Further Research Questions

- Is CS student retention affected by the human language of the code?
- What code documentation is most effective for non-English speakers?
- How can we increase support for non-English languages in current IDEs?